Guía de Estudio: Haskell y los Fundamentos del Paradigma Funcional

Esta guía articula la sintaxis fundamental de **Haskell** con los conceptos nucleares del paradigma funcional.

1. Fundamentos de Haskell: Sintaxis, Evaluación y Memoria

Haskell utiliza una sintaxis limpia inspirada directamente en el **cálculo lambda** y la notación matemática. Haskell es un lenguaje funcional puro, con tipado estático fuerte e inferido, y evaluación diferida o perezosa (*lazy evaluation*).

Evaluación Diferida (*Lazy Evaluation* o *Call-by-Need*)

Haskell no evalúa los argumentos de una función al momento de ser invocada (como sí lo hace la evaluación ansiosa o *call-by-value*). En su lugar, posterga el cómputo hasta que el resultado sea estrictamente necesario para la ejecución:

- Permite definir y manipular **estructuras y listas infinitas** sin provocar desbordamientos de memoria ni bucles infinitos, ya que solo se calcula la porción requerida.
- Desacopla la generación de datos de su consumo real.

```
-- Generador de una lista infinita de enteros a partir de 1
enteros :: [Integer]
enteros = [1..]

-- Solo se calculan los primeros 5 elementos al consumirlos
primerosCinco :: [Integer]
primerosCinco = take 5 enteros  -- Resultado: [1, 2, 3, 4, 5]
```

Firmas y Declaración de Tipos (::)

En Haskell, las funciones y los valores poseen tipos definidos en tiempo de compilación. Aunque el compilador puede inferirlos, la convención didáctica y formal exige declarar la firma de tipo separando los argumentos mediante la flecha (->), reflejando el principio de currificación (currying):

```
-- Declaración de tipo (firma): toma dos enteros y produce un entero
sumar :: Int -> Int -> Int
sumar a b = a + b
```

Entrada y Salida con Pureza Referencial (Mónada IO)

Haskell prohíbe funciones impuras que alteren la memoria de forma arbitraria o produzcan efectos colaterales descontrolados en pantalla. Toda acción de entrada/salida queda encapsulada dentro de la **mónada IO**, la cual modela una

acción que interactúa con el mundo exterior manteniendo el resto del código puramente referencial:

```
-- putStrLn imprime una cadena y añade un salto de línea
-- print imprime cualquier dato con formato legible
main :: IO ()
main = do
   putStrLn "Hola mundo desde Haskell!"
   print (sumar 10 5)
```

Ámbito y Ligaduras Locales (let y where)

En Haskell no existen variables globales mutables reasignables (a diferencia de defparameter o defvar en LISP). Los valores son **inmutables** y se asocian mediante ligaduras léxicas locales acotadas a la expresión:

```
-- 1. Ligadura local con 'let ... in'
areaTrianguloLet :: Double -> Double -> Double
areaTrianguloLet base altura =
   let producto = base * altura
   in producto / 2

-- 2. Cláusula 'where' (declara definiciones auxiliares al final de la función)
areaTrianguloWhere :: Double -> Double -> Double
areaTrianguloWhere base altura = producto / 2
   where producto = base * altura
```

Bifurcaciones Condicionales: if-then-else, Guardas y *Pattern Matching*

Haskell no tiene sentencias de control imperativas; cada bifurcación es una expresión matemática que debe devolver un valor en todas sus ramas:

```
-- 1. Expresión if-then-else (ambas ramas son obligatorias y del mismo tipo)
esMayorDeEdad :: Int -> String
esMayorDeEdad edad = if edad >= 18 then "Mayor de edad" else "Menor de edad"

-- 2. Guardas con '|' (análogo a 'cond' en LISP)
clasificarNota :: Int -> String
clasificarNota nota
   | nota >= 19 = "Excelente"
   | nota >= 14 = "Aprobado"
   | nota >= 11 = "Suficiente"
   | otherwise  = "Reprobado"  -- 'otherwise' actúa como el caso por defecto (t en LISP)
```

Funciones Anónimas (Expresiones Lambda \)

Se definen utilizando la barra invertida \ (que emula la letra griega λ), seguida de los parámetros y la flecha ->:

```
-- Función anónima para calcular el área de un rectángulo aplicada de inmediato:
areaRectangulo :: Int
```

```
areaRectangulo = (\base altura -> base * altura) 10 5  -- Evalúa a 50
```

2. Anatomía de las Listas en Haskell

Las listas en Haskell son colecciones homogéneas (todos sus elementos deben ser del mismo tipo) y están construidas recursivamente mediante dos constructores fundamentales:

- []: Lista vacía (equivalente a NIL en LISP o nil en Hope).
- : (Operador *cons*): Añade un elemento a la cabeza de una lista existente.

```
-- La lista [10, 20, 30] es una abreviación de: 10 : (20 : (30 : []))
miLista :: [Int]
miLista = [10, 20, 30]
```

Operaciones Estructurales y Descomposición:

- **head** (Cabeza / *car* en LISP): Extrae el primer elemento.
- **tail** (Cola / *cdr* en LISP): Devuelve el resto de la lista sin su cabeza.
- **null**: Predicado que devuelve True si la lista está vacía ([]), indispensable para identificar casos base recursivos.

```
primerElemento = head [10, 20, 30]  -- 10
restoLista     = tail [10, 20, 30]  -- [20, 30]
estaVacia      = null []            -- True
```

3. Los 5 Pilares del Paradigma Funcional en Haskell

Pilar 1: Transparencia Referencial y Funciones Puras

Una función pura carece de efectos laterales y satisface la propiedad de que, ante los mismos argumentos de entrada, siempre devolverá exactamente la misma salida. Esto permite sustituir cualquier llamada por su valor sin alterar el comportamiento global del sistema (*razonamiento ecuacional*).

```
-- Puro: determinista y sin efectos secundarios
sumarPuro :: Int -> Int -> Int
sumarPuro a b = a + b
```

Pilar 2: Ausencia de Estado Mutable y Recursividad

En Haskell no existen celdas de memoria reasignables (x = x + 1) ni bucles de control imperativo (for, while). El flujo se gestiona mediante **recursión pura** y **emparejamiento de patrones (*pattern matching*)**:

```
-- Recorrido recursivo con pattern matching sobre una lista
sumarLista :: [Int] -> Int
sumarLista []     = 0                -- Caso base: lista vacía
sumarLista (x:xs) = x + sumarLista xs   -- Caso recursivo: cabeza + llamada sobre cola
```

Patrón con acumulador (recursión de cola / función auxiliar):

```
-- Retorna el primer valor par y su posición (1-based)
primerParPos :: [Int] -> Maybe (Int, Int)
primerParPos lista = helper lista 1
  where
    helper [] _ = Nothing
    helper (x:xs) pos
        | even x    = Just (x, pos)
        | otherwise = helper xs (pos + 1)
```

Pilar 3: Funciones como Ciudadanos de Primera Clase y Orden Superior

Las funciones pueden asignarse a variables, pasarse como argumentos y devolverse como resultados de cómputo.

- **Currificación:** En Haskell toda función toma formalmente un solo argumento a la vez y devuelve otra función hasta agotar los parámetros.
- **map:** Aplica una función transformadora sobre cada elemento de una lista produciendo una nueva lista.
- **filter:** Selecciona únicamente los elementos que cumplen un predicado booleano.
- **foldr** / **foldl** (Reducción / *reduce*): Combina los elementos de una lista mediante una función binaria hasta reducirlos a un único valor.

```
-- Aplicación de función de orden superior:
cuadrados :: [Int]
cuadrados = map (\x -> x * x) [1, 2, 3, 4]  -- [1, 4, 9, 16]

pares :: [Int]
pares = filter even [1, 2, 3, 4, 5, 6]      -- [2, 4, 6]

sumatoria :: Int
sumatoria = foldr (+) 0 [1, 4, 9, 16]       -- 30
```

Pilar 4: Composición Funcional (.)

Permite encadenar transformaciones de datos directamente, de modo que la salida de una función alimenta la entrada de la siguiente sin necesidad de almacenar variables intermedias mutables:

$$(f \circ g)(x) = f\big(g(x)\big)$$

```
-- Elevar al cuadrado los números de una lista y sumarlos mediante composición
sumaCuadrados :: [Int] -> Int
sumaCuadrados = sum . map (\x -> x * x)

-- Uso: sumaCuadrados [2, 3, 4] -> 4 + 9 + 16 = 29
```

Pilar 5: Polimorfismo Paramétrico y Clases de Tipos (*Typeclasses*)

Haskell proporciona **polimorfismo paramétrico** estricto mediante variables de tipo (`a`, `b`), permitiendo escribir funciones genéricas independientes de la representación de los datos:

```
-- Extrae los primeros n elementos de cualquier lista (polimórfica en 'a')
tomarN :: Int -> [a] -> [a]
tomarN n _  | n <= 0 = []
tomarN _ []          = []
tomarN n (x:xs)      = x : tomarN (n - 1) xs
```

Cuando una operación exige ciertas capacidades sobre el tipo (por ejemplo, comparar igualdad o realizar operaciones aritméticas), Haskell utiliza **Clases de Tipos (*Typeclasses*)** para lograr sobrecarga controlada:

```
-- 'Eq a' restringe 'a' a tipos que implementan igualdad (==)
pertenece :: Eq a => a -> [a] -> Bool
pertenece _ []     = False
pertenece e (x:xs) = (e == x) || pertenece e xs
```

4. Encapsulación de Efectos: El Concepto de Mónada en Haskell

Haskell formaliza el manejo de secuencias de cómputo y efectos laterales a través del concepto de **Mónada**. Una mónada encapsula un valor en un contexto computacional sin vulnerar la transparencia referencial ni la pureza funcional:

1. **Constructor de tipo unario (`m`):** Envuelve un dato de tipo `a` en un contexto monádico `m a` (por ejemplo, `IO a`, `Maybe a`, `[a]`).
2. **`return`:** Inyecta un valor ordinario en la mónada (`return :: a -> m a`).
3. **Operador de enlace (*bind*, `>>=`):** Encadena dos cómputos monádicos, extrayendo el resultado del primero y pasándolo como entrada al segundo:

$$\texttt{>>=} :: m\ a \rightarrow (a \rightarrow m\ b) \rightarrow m\ b$$

```
-- Lectura y encadenamiento monádico explícito con (>>=):
leerEImprimir :: IO ()
leerEImprimir =
  getChar >>= \c1 ->
  getChar >>= \c2 ->
  putChar c2 >>= \_ ->
  putChar c1
```

Notación do

Haskell provee azúcar sintáctico para evitar el encadenamiento extenso de operadores >>=, permitiendo escribir código secuencial legible sin perder la pureza funcional de fondo:

```
conversacion :: IO ()
conversacion = do
   putStrLn "Ingrese su nombre:"
   nombre <- getLine
   putStrLn ("Bienvenido a Haskell, " ++ nombre ++ "!")
```

5. Tabla de Equivalencias: Imperativo vs. Common LISP vs. Haskell

Concepto	**Enfoque Imperativo (C / C++)**	**Common LISP**	**Haskell**
Flujo de Ejecución	Sentencias secuenciales, bucles for, while	Expresiones recursivas / bucles híbridos (loop, dotimes)	Evaluación de expresiones recursivas puras y evaluación diferida (*lazy*)
Variables	Celdas de memoria mutables reasignables (x++)	Ligaduras léxicas (let), variables especiales mutables (setf)	Ligaduras inmutables puras (let, where)
Sistema de Tipos	Estático y comprobado por compilador (débil/medio en C)	Dinámico (verificación de tipos en tiempo de ejecución)	Estático, fuerte e inferido (sistema Hindley-Milner)
Colecciones	Arreglos indexados arr[i]	Celdas *cons* enlazadas (car, cdr)	Listas algebraicas homogéneas (head, tail, :) e infinitas
Transformación de Lotes	Bucles manuales con acumuladores mutables	mapcar y reduce nativos sobre listas	map, filter, foldr y composición (.)
Manejo de Entrada/Salida	Sentencias impuras directas (printf, cin)	Salidas impuras directas (print, format t)	Aislada y encapsulada

Concepto	Enfoque Imperativo (C / C++)	Common LISP	Haskell
			estrictamente en la Mónada IO